COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S1
DATE PERFORMED:	Jul 16, 2026	**DATE SUBMITTED:**	Jul 21, 2026
NAME:	Jave Joaquin Obias	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 2.1 **[Arrays, Pointers and Dynamic Memory Allocation]**

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

In this activity, we will explore static and dynamic memory allocation through by utilizing arrays and other components mentioned beforehand, this will be for the implementation of a student list with the students' names and age. To begin, we will include pertinent libraries for input/output stream and strings (which we will be used in this activity).

```
#include <iostream>
#include <string.h>
```

Now we will create the student class with private attributes studentName and studentAge.

```
class Student{
private:
  std::string studentName;
  int studentAge;
```

Then, as discussed beforehand, we have to define the constructor and the big three: the destructor, copy constructor and copy assignment operator. We will assign default values for the parameters of our constructor in the event that it is uninitialized.

```
public:
  //Constructor
  Student(std::string newName ="John Doe", int newAge=18){
    studentName = std::move(newName);
    studentAge = newAge;
    std::cout << "Constructor Called." << std::endl;
  };
```

The big three is then defined. We will add an output stream for each to observe when each of the following functions are implicitly or explicitly called.

```
  //Deconstructor
  ~Student(){
    std::cout << "Destructor Called." << std::endl;
  }
  //Copy Constructor
  Student(const Student &copyStudent){
    std::cout << "Copy Constructor Called" << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
  }
```

```
    //Display Attributes
    void printDetails(){
      std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};
```

The driver program is now to be defined. We want to utilize the main function to simply show the static and dynamic allocation. However, we will explore the initial goal of creating instances of our Student class.

```
int main() {
  Student student1("Roman", 28);
  Student student2(student1);
  Student student3;
  student3 = student2;
  return 0;
}
```

Run the code and show the output. Include the screenshot in table 2-1 followed by your observation.

Now, modify the driver program so that we have the array size of 5, an array of Student objects, the list of students' names and their age.

```
int main() {
  const size_t j = 5;
  Student studentList[j] = {};
  std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
  int ageList[j] = {15, 16, 18, 19, 16};
  return 0;
}
```

Run the code with the modified driver function and show the output. Include the screenshot in table 2-2 followed by your observation.

We have so far created static memory allocation through the use of the arrays. We will now dynamically allocate instances of the student class and store the newly created objects in the locations pointed to by our array.

```
int main() {
  const size_t j = 5;
  Student studentList[j] = {};
  std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
  int ageList[j] = {15, 16, 18, 19, 16};
for(int i = 0; i < j; i++){ //loop A
  Student *ptr = new Student(namesList[i], ageList[i]);
  studentList[i] = *ptr;
}
for(int i = 0; i < j; i++){ //loop B
  studentList[i].printDetails();
}
return 0;
}
```

Discuss what is done by loop A and loop B in table 2-3. Additionally, discuss the output and whether the functions are working as intended. If any corrections were made, further provide your modification and analysis in table 2-4.

Screenshot

```cpp
#include <iostream>
#include <string.h>

class student{
    private:
        std::string studentname;
        int studentage;

    public:
        //Constructor
        student(std::string newname = "John Doe", int newage = 18){
            studentname = std::move(newname);
            studentage = newage;
            std::cout << "Constructor was called." << std::endl;
        }
```

```cpp
    //Deconstructor
    ~student(){
        std::cout << "Destructor was called." << std::endl;
    }

    //Copy Constructor
    student(const student &copystudent){
        std::cout << "Copy constructor was called." << std::endl;
        studentname = copystudent.studentname;
        studentage = copystudent.studentage;
    }

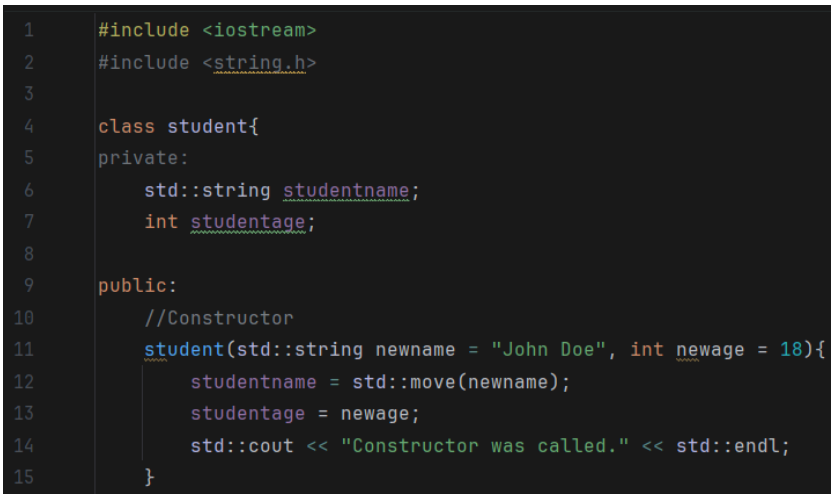
    //Display Attributes
    void printdetails(){
        std::cout << this->studentname << " " << this->studentage << std::endl;
    }
```

```cpp
int main(){
    student stud1( newname: "Jave",  newage: 19);
    student stud2( stud1);
    student stud3;
    stud3 = stud2;

    return 0;
}
```

Output	
Observation	I observed that stud2 copied the values of stud1 because the Copy Constructor was called after stud1 was initialized using the Constructor. In stud3, it first displayed the default values of studentname and studentage, then after the assignment stud3 = stud2, it changed its values to be the same as stud2. Lastly, the Destructor was called three times because there were three objects created in the program.

Table 2-1. Initial Driver Program

Screenshot	

```
    //Deconstructor
    ~student(){
        std::cout << "Destructor was called." << std::endl;
    }

    //Copy Constructor
    student(const student &copystudent){
        std::cout << "Copy constructor was called." << std::endl;
        studentname = copystudent.studentname;
        studentage = copystudent.studentage;
    }

    //Display Attributes
    void printdetails(){
        std::cout << this->studentname << " " << this->studentage << std::endl;
    }
```

```
int main(){
    const size_t j = 5;
    student studentlist[j] = {};
    std::string namelist[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int agelist[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

Output

```
C:\Users\javej\OneDrive\Documents\OBIAS\cmake-build-debug\CPE010_HOA2_ILOA2_OBIAS.exe
Constructor was called.
Constructor was called.
Constructor was called.
Constructor was called.
Constructor was called.
Destructor was called.
Destructor was called.
Destructor was called.
Destructor was called.
Destructor was called.

Process finished with exit code 0
```

Observation

I observed that the program created five student objects using the default constructor because the studentlist array was initialized with empty braces. The namelist and agelist arrays stored the names and ages, but they were not yet assigned to the objects in studentlist, so all the students still had the default values. Lastly, when the program ended, the Destructor was called five times because there were five student objects created in the array.

Table 2-2. Modified Driver Program with Student Lists

Screenshot

```cpp
#include <iostream>
#include <string.h>

class student{
private:
    std::string studentname;
    int studentage;

public:
    //Constructor
    student(std::string newname = "John Doe", int newage = 18){
        studentname = std::move(newname);
        studentage = newage;
        std::cout << "Constructor was called." << std::endl;
    }
```

```cpp
    //Deconstructor
    ~student(){
        std::cout << "Destructor was called." << std::endl;
    }

    //Copy Constructor
    student(const student &copystudent){
        std::cout << "Copy constructor was called." << std::endl;
        studentname = copystudent.studentname;
        studentage = copystudent.studentage;
    }

    //Display Attributes
    void printdetails(){
        std::cout << this->studentname << " " << this->studentage << std::endl;
    }
```

```cpp
int main() {
    const size_t j = 5;
    student studentList[j] = {};
    std::string namesList[j] = { "Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++){ //loop A
        student *ptr = new student( namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }

    for(int i = 0; i < j; i++){ //loop B
        studentList[i].printdetails();
    }

    return 0;
}
```

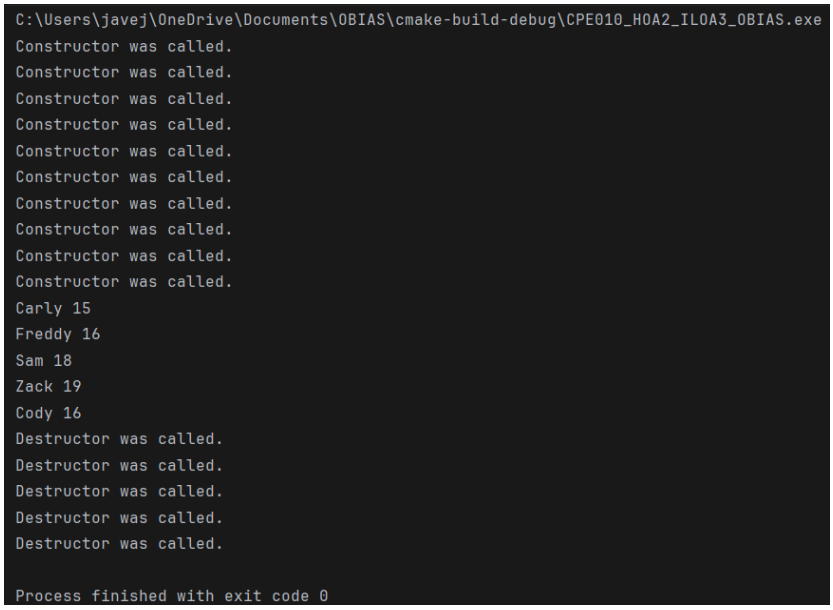
Output	
Observation	I observed that the program first created five student objects with the default constructor because studentList was initialized with empty braces. In Loop A, each object was assigned the corresponding name and age from namesList and ageList by creating a temporary object and copying its values into studentList. In Loop B, the program displayed the updated details of all five students, and when the program ended, the Destructor was called for every object that was created.

Table 2-3. Final Driver Program

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenna's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.
Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem

description above.
Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.
Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.
Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.
Problem 5: Discuss the

```cpp
#include <iostream>
#include <string>

class Product {
protected:
    std::string itemName;
    double itemPrice;
    int itemCount;

public:
    Product(std::string name, double price, int count)
        : itemName(name), itemPrice(price), itemCount(count) {}

    virtual ~Product() {}

    double getAmount() const {
        return itemPrice * itemCount;
    }

    std::string getName() const {
        return itemName;
    }

    virtual std::string getType() const = 0;

    void showInfo() const {
        std::cout << getType() << " : " << itemName
                  << " | Price: " << itemPrice
                  << " | Count: " << itemCount
                  << " | Total: " << getAmount()
                  << std::endl;
    }
};

class Drink : public Product {
public:
    Drink(std::string name, double price, int count)
        : Product(name, price, count) {
        std::cout << "Added drink: " << itemName << std::endl;
    }

    ~Drink() override {
        std::cout << "Removing drink: " << itemName << std::endl;
    }

    std::string getType() const override {
```

```
        return "Drink";
    }
};

class Snack : public Product {
public:
    Snack(std::string name, double price, int count)
        : Product(name, price, count) {
        std::cout << "Added snack: " << itemName << std::endl;
    }

    ~Snack() override {
        std::cout << "Removing snack: " << itemName << std::endl;
    }

    std::string getType() const override {
        return "Snack";
    }
};

double computeTotal(Product* products[], int size) {
    double sum = 0;

    for (int i = 0; i < size; i++) {
        sum += products[i]->getAmount();
    }

    return sum;
}

int main() {
    Product* storeItems[4];

    storeItems[0] = new Drink("Cola", 25, 3);
    storeItems[1] = new Drink("Juice", 30, 2);
    storeItems[2] = new Snack("Chips", 18, 5);
    storeItems[3] = new Snack("Cookies", 20, 4);

    int totalItems = 4;

    std::cout << "\nStore Items\n\n";

    for (int i = 0; i < totalItems; i++) {
        storeItems[i]->showInfo();
    }

    std::cout << "\nOverall Total: " << computeTotal(storeItems, totalItems) <<
std::endl;

    std::cout << "\nDeleting Cookies...\n";

    for (int i = 0; i < totalItems; i++) {
        if (storeItems[i]->getName() == "Cookies") {
            delete storeItems[i];
```

```
            storeItems[i] = storeItems[totalItems - 1];
            totalItems--;
            break;
        }
    }

    std::cout << "\nUpdated List\n\n";

    for (int i = 0; i < totalItems; i++) {
        storeItems[i]->showInfo();
    }

    std::cout << "\nNew Total: " << computeTotal(storeItems, totalItems) <<
std::endl;

    for (int i = 0; i < totalItems; i++) {
        delete storeItems[i];
    }

    return 0;
}
```

```
C:\Users\javej\OneDrive\Documents\OBIAS\cmake-build-debug\CPE010_ILOC_OBIAS.exe
Added drink: Cola
Added drink: Juice
Added snack: Chips
Added snack: Cookies

Store Items

Drink : Cola | Price: 25 | Count: 3 | Total: 75
Drink : Juice | Price: 30 | Count: 2 | Total: 60
Snack : Chips | Price: 18 | Count: 5 | Total: 90
Snack : Cookies | Price: 20 | Count: 4 | Total: 80

Overall Total: 305

Deleting Cookies...
Removing snack: Cookies

Updated List

Drink : Cola | Price: 25 | Count: 3 | Total: 75
Drink : Juice | Price: 30 | Count: 2 | Total: 60
Snack : Chips | Price: 18 | Count: 5 | Total: 90
```

```
New Total: 225
Removing drink: Cola
Removing drink: Juice
Removing snack: Chips
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Python Tutor**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Visualizing the code**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use: **I used Python Tutor for understand what happened on the code**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.